Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота № 4

« Аналітичні SQL-запити (OLAP) »

Роботу виконали

Студент групи ІО-45

Соловійщук

Назар

Роботу перевірив:

Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 4

Тема: Аналітичні SQL-запити (OLAP)

Мета:

Навчитися використовувати агрегатні функції (COUNT, SUM, AVG, MIN, MAX) для обчислення статистики.
Опанувати групування даних за допомогою GROUP BY та фільтрацію агрегованих результатів через HAVING.
Навчитися об'єднувати дані з кількох таблиць за допомогою різних типів JOIN (INNER, LEFT, CROSS) для створення аналітичних звітів.
Практикувати використання підзапитів у різних частинах інструкції SELECT

SQL-скрипт

```
-- ==========================================
-- ЛАБОРАТОРНА РОБОТА №4
-- Студент: Соловійщук Назар (група ІО-45)
-- Тема: Аналітичні SQL-запити (OLAP)
-- ==========================================

-- ЧАСТИНА 1: АГРЕГАЦІЯ ТА ГРУПУВАННЯ (Мінімум 4 запити) [cite: 576]

-- 1. Базова агрегація (COUNT)
-- Підрахувати загальну кількість фізичних копій дисків у системі
SELECT COUNT(*) AS total_disks
FROM inventory; [cite: 580, 582]

-- 2. Агрегація (AVG, MAX, MIN)
-- Проаналізувати тривалість фільмів: середня, максимальна та мінімальна
SELECT
   AVG(duration) AS avg_duration,
    MAX(duration) AS max_movie,
    MIN(duration) AS min_movie
FROM movies; [cite: 584]

-- 3. Групування (GROUP BY)
-- Підрахувати кількість копій (дисків) для кожного фільму
SELECT movie_id, COUNT(*) AS copies_count
FROM inventory
GROUP BY movie_id; [cite: 585, 592]

-- 4. Фільтрування груп (HAVING)
-- Знайти клієнтів, які здійснили більше ніж 1 оренду (активні користувачі)
SELECT customer_id, COUNT(*) AS rental_count
FROM rentals
GROUP BY customer_id
HAVING COUNT(*) > 1; [cite: 571, 599]
```

```sql
-- ЧАСТИНА 2: ОБ'ЄДНАННЯ ТАБЛИЦЬ (JOINs) (Мінімум 3 запити) [cite: 576]

-- 5. INNER JOIN
-- Список: Ім'я клієнта, Назва фільму та сума проведеного платежу
SELECT
   c.first_name || ' ' || c.last_name AS Клієнт,
   m.title AS Фільм,
   p.amount AS Сума_оплати
FROM payments p
INNER JOIN rentals r ON p.rental_id = r.rental_id
INNER JOIN customers c ON r.customer_id = c.customer_id
INNER JOIN inventory i ON r.inventory_id = i.inventory_id
INNER JOIN movies m ON i.movie_id = m.movie_id; [cite: 601, 606]

-- 6. LEFT JOIN
-- Показати всі фільми та загальну кількість їх оренд (включаючи ті, що не
орендувались)
SELECT
   m.title,
   COUNT(r.rental_id) AS rental_count
FROM movies m
LEFT JOIN inventory i ON m.movie_id = i.movie_id
LEFT JOIN rentals r ON i.inventory_id = r.inventory_id
GROUP BY m.title; [cite: 607, 608]

-- 7. CROSS JOIN
-- Матриця всіх можливих комбінацій клієнтів та існуючих жанрів фільмів
SELECT
   c.first_name,
   m.genre
FROM customers c
CROSS JOIN (SELECT DISTINCT genre FROM movies) m;


-- ЧАСТИНА 3: ПІДЗАПИТИ (SUBQUERIES) (Мінімум 3 запити) [cite: 576]

-- 8. Підзапит у WHERE (Оператор IN)
-- Знайти імена клієнтів, які мають хоча б один запис про оренду
SELECT first_name, last_name
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id
   FROM rentals
); [cite: 576]

-- 9. Підзапит у SELECT
-- Вивести назву фільму та кількість його фізичних копій (підрахунок підзапитом)
SELECT
   title,
   (SELECT COUNT(*) FROM inventory WHERE inventory.movie_id = movies.movie_id) AS
кількість_копій
FROM movies; [cite: 576]
```

```
-- 10. Підзапит з агрегацією (WHERE + Scalar Subquery)
-- Знайти фільми, тривалість яких більша за середню тривалість усіх фільмів у базі
SELECT title, duration
FROM movies
WHERE duration > (
    SELECT AVG(duration) FROM movies
); [cite: 576]
```

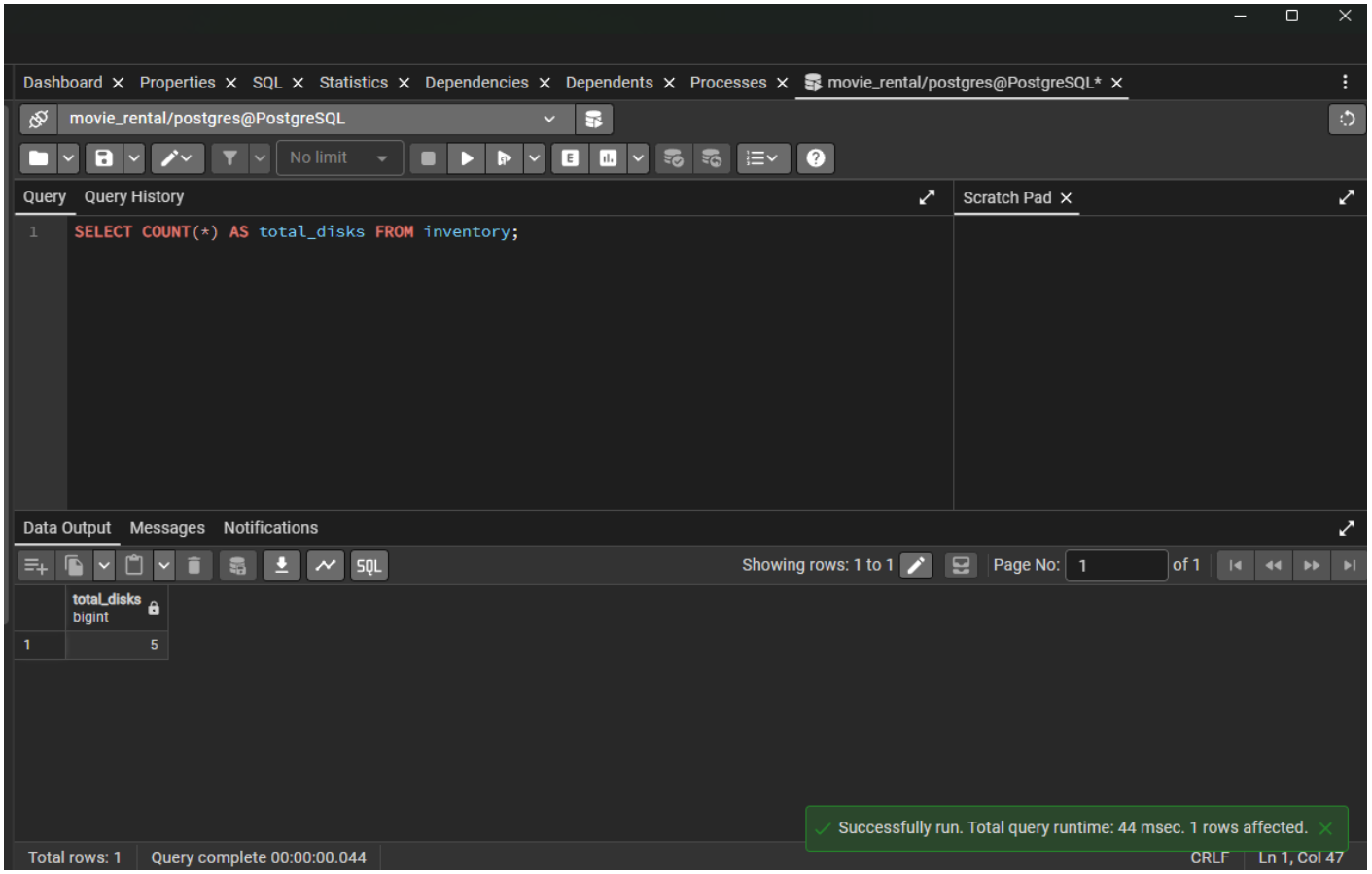
2. Опис аналітичних запитів
Частина 1: Агрегація та групування
У цій частині реалізовано 4 запити:

1 COUNT: Підрахунок загальної кількості дисків в інвентарі.

2 Статистика тривалості: Використання AVG, MAX та MIN для аналізу хронометражу фільмів.

3 GROUP BY: Групування інвентарю за ідентифікатором фільму для визначення кількості копій кожної стрічки.

4 HAVING: Пошук активних клієнтів, які здійснили більше ніж одну оренду.



Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT
    AVG(duration) AS avg_duration,
    MAX(duration) AS max_movie,
    MIN(duration) AS min_movie
FROM movies;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	avg_duration numeric	max_movie integer	min_movie integer
1	156.4000000000000000	175	136

Successfully run. Total query runtime: 47 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.047 CRLF Ln 5, Col 13

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT movie_id, COUNT(*) AS copies_count
FROM inventory
GROUP BY movie_id;
```

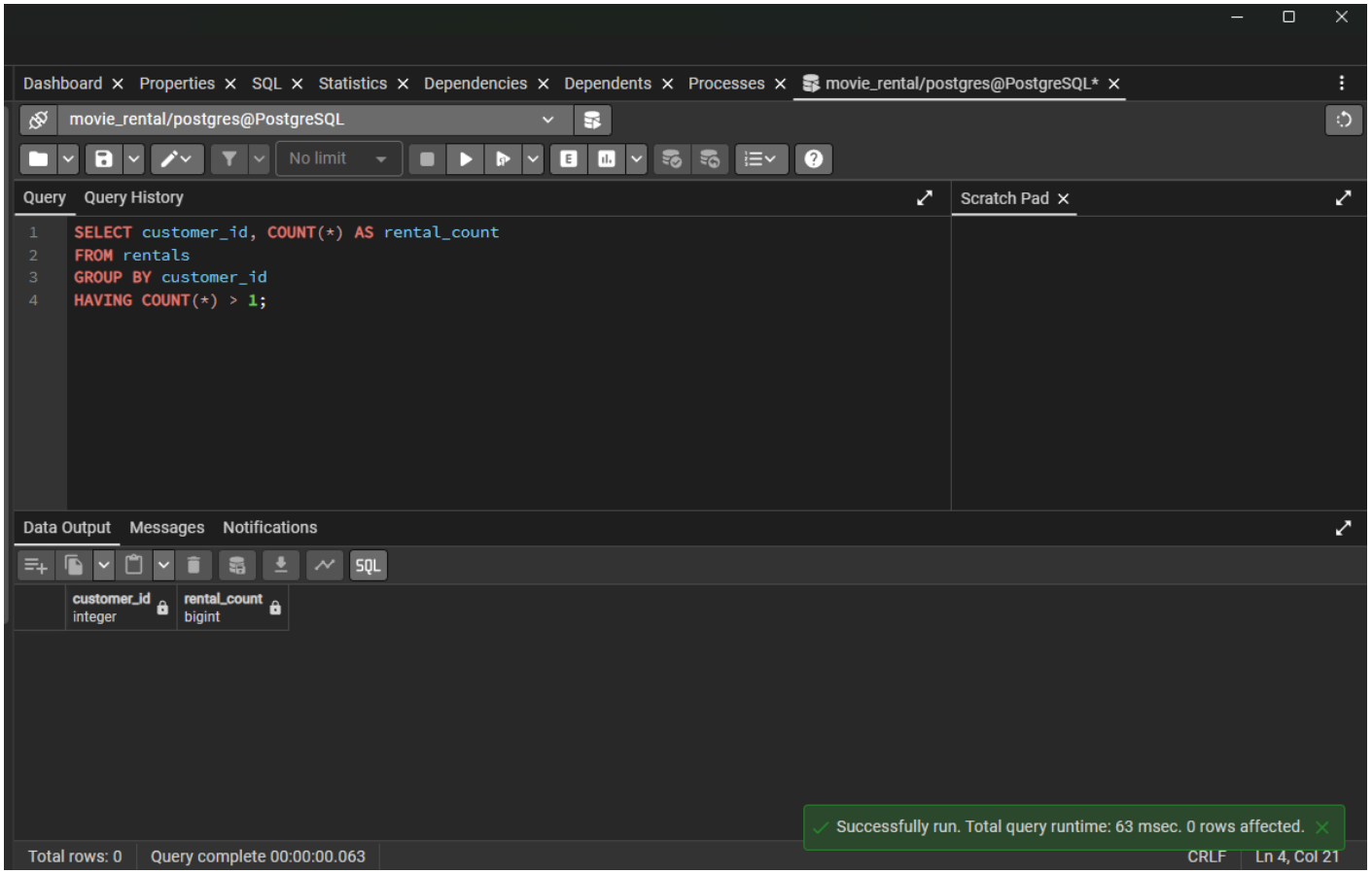
Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	movie_id integer	copies_count bigint
1	3	1
2	4	1
3	2	1
4	1	2

Successfully run. Total query runtime: 54 msec. 4 rows affected.

Total rows: 4 Query complete 00:00:00.054 CRLF Ln 3, Col 19



Частина 2: Об'єднання таблиць (JOINs)
Реалізовано 3 типи об'єднань:

1 INNER JOIN: Складений запит для виведення імен клієнтів разом із назвами фільмів та сумами їхніх платежів .

2 LEFT JOIN: Аналіз популярності всіх фільмів (навіть тих, що ще не були в оренді).

3 CROSS JOIN: Формування матриці всіх можливих комбінацій клієнтів та жанрів фільмів для аналізу ринку.

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT
    c.first_name || ' ' || c.last_name AS Клієнт,
    m.title AS Фільм,
    p.amount AS Сума_оплати
FROM payments p
INNER JOIN rentals r ON p.rental_id = r.rental_id
INNER JOIN customers c ON r.customer_id = c.customer_id
INNER JOIN inventory i ON r.inventory_id = i.inventory_id
INNER JOIN movies m ON i.movie_id = m.movie_id;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	Клієнт text	Фільм character varying (255)	Сума_оплати numeric (10,2)
1	Nazar Solovishch...	Inception	50.00
2	Nazar Solovishch...	Inception	5.00
3	Alice Smith	The Godfather	45.00

Successfully run. Total query runtime: 42 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.042 CRLF Ln 9, Col 48

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT
    m.title,
    COUNT(r.rental_id) AS rental_count
FROM movies m
LEFT JOIN inventory i ON m.movie_id = i.movie_id
LEFT JOIN rentals r ON i.inventory_id = r.inventory_id
GROUP BY m.title;
```

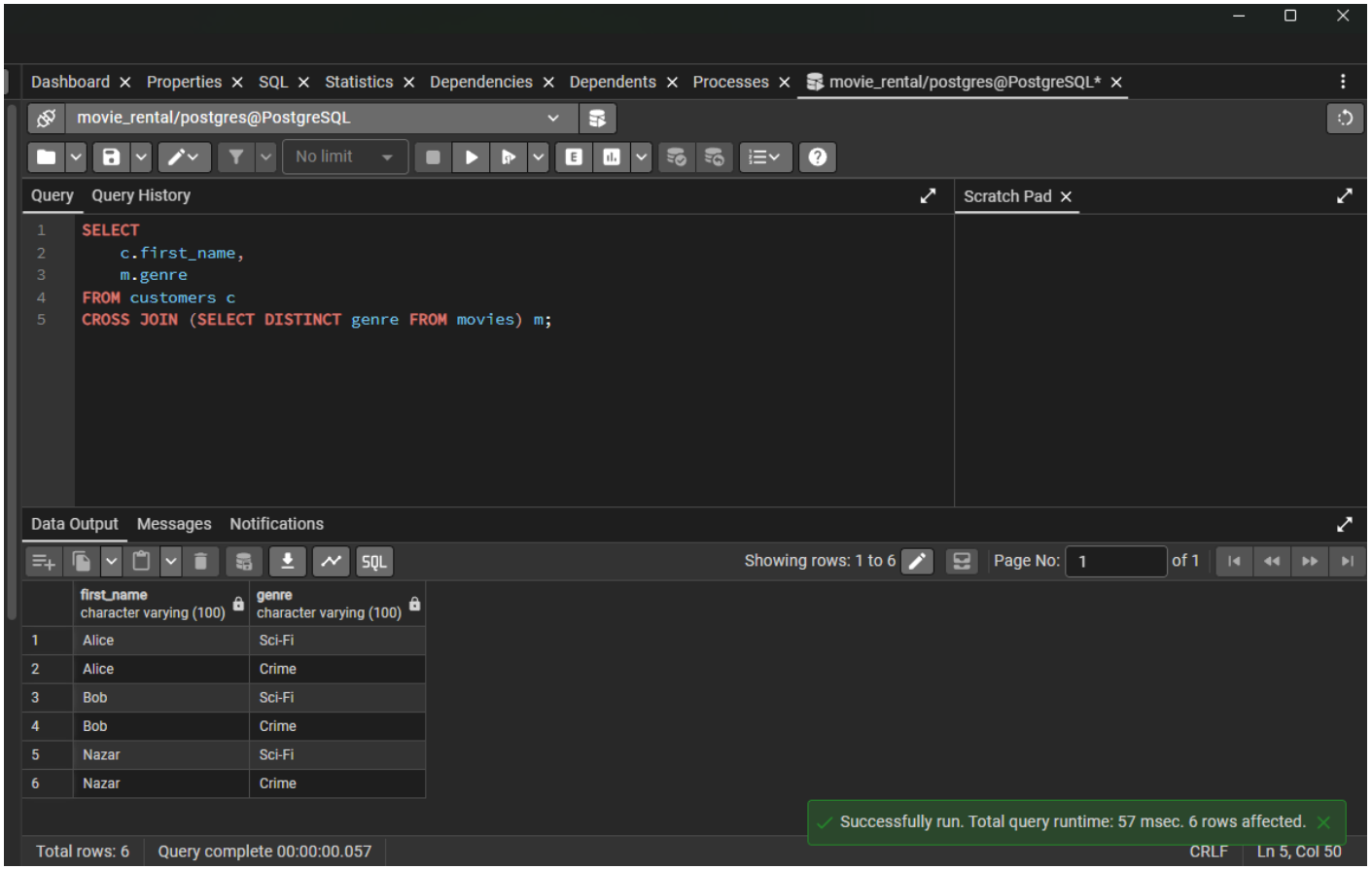
Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	title character varying (255)	rental_count bigint
1	The Matrix	0
2	Inception	1
3	Pulp Fiction	1
4	The Godfather	1
5	Interstellar	0

Successfully run. Total query runtime: 70 msec. 5 rows affected.

Total rows: 5 Query complete 00:00:00.070 CRLF Ln 7, Col 18



Частина 3: Підзапити (Subqueries)
Реалізовано 3 варіанти підзапитів:

1 Підзапит у WHERE: Вибірка клієнтів, чиї ID існують у таблиці оренд.

2 Підзапит у SELECT: Динамічне обчислення кількості дисків для кожного фільму безпосередньо в основному запиті.

3 Скалярний підзапит: Пошук фільмів, тривалість яких перевищує середній показник по всій базі.

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT first_name, last_name
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id
    FROM rentals
);
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	first_name character varying (100)	last_name character varying (100)
1	Bob	Johnson
2	Alice	Smith
3	Nazar	Solovishchuk

Successfully run. Total query runtime: 56 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.056 CRLF Ln 6, Col 3

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × movie_rental/postgres@PostgreSQL* ×

movie_rental/postgres@PostgreSQL

No limit

Query Query History

Scratch Pad ×

```sql
SELECT
    title,
    (SELECT COUNT(*) FROM inventory WHERE inventory.movie_id = movies.movie_id) AS кількіст
FROM movies;
```

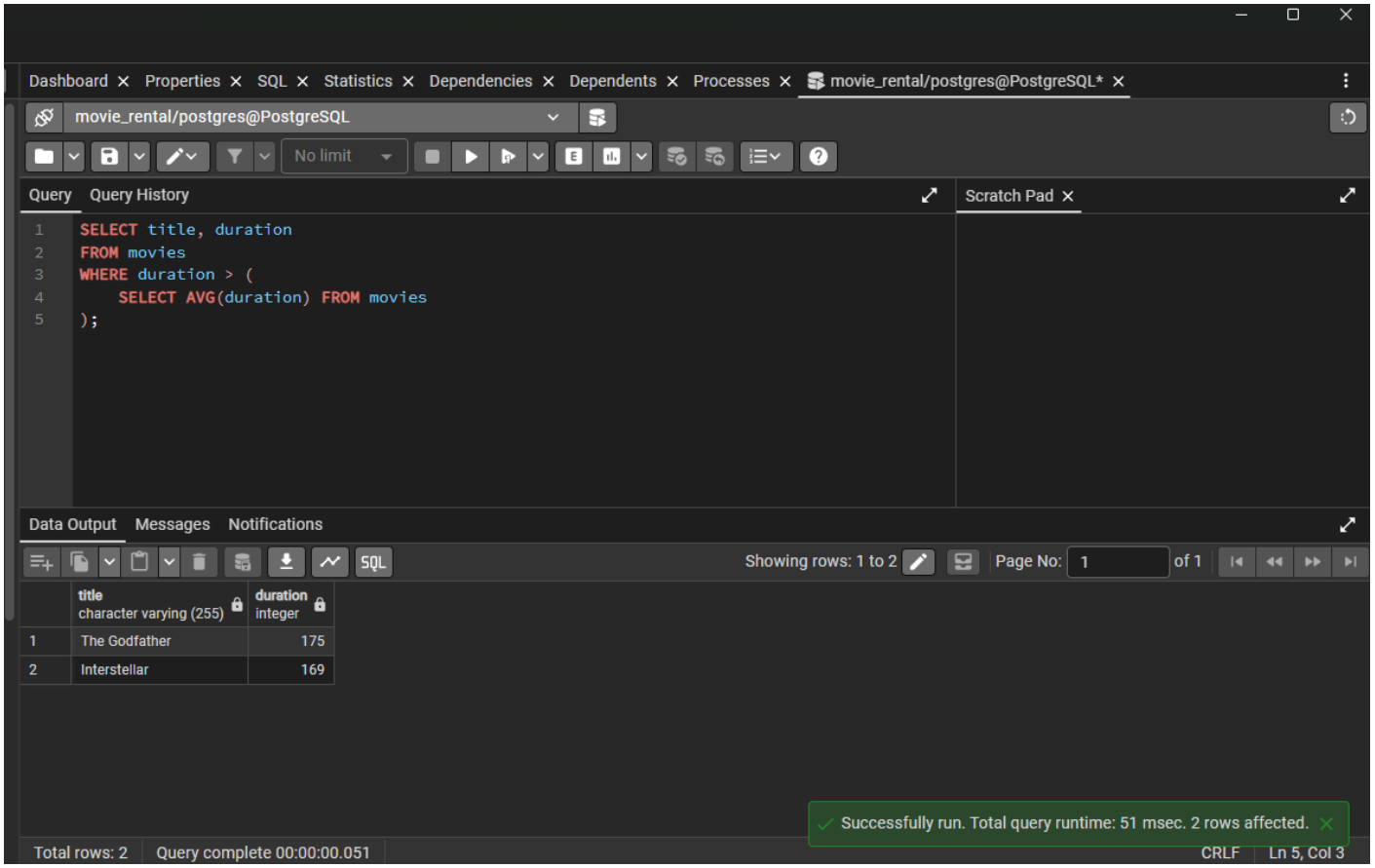
Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	title character varying (255)	кількість_копій bigint
1	Inception	2
2	The Godfather	1
3	Pulp Fiction	1
4	The Matrix	1
5	Interstellar	0

Successfully run. Total query runtime: 51 msec. 5 rows affected.

Total rows: 5 Query complete 00:00:00.051 CRLF Ln 4, Col 13



Висновок:

У ході лабораторної роботи було опановано інструменти онлайн-аналітичної обробки даних (OLAP) у PostgreSQL. Було доведено, що використання агрегатних функцій у поєднанні з групуванням дозволяє швидко отримувати статистичні звіти. Практичне застосування різних типів `JOIN` продемонструвало можливість гнучкого об'єднання даних з багатьох таблиць. Використання підзапитів дозволило реалізувати складну логіку фільтрації, що базується на розрахункових показниках. Всі запити були протестовані та повернули коректні результати, що підтверджує цілісність та правильність структури бази даних.

Висновок:
У ході лабораторної роботи було вивчено основні операції маніпулювання даними (DML) у середовищі PostgreSQL. Практично перевірено, як оператори `INSERT`, `UPDATE` та `DELETE` впливають на вміст таблиць. Особливу увагу було приділено використанню речення `WHERE` для запобігання ненавмисної зміни або видалення всіх даних у таблиці. Також було підтверджено роботу обмежень зовнішніх ключів, які блокують видалення батьківських записів, доки існують пов'язані дочірні записи